



Universidad Nacional Experimental de Guayana

Coordinación General de Pregrado

Vicerrectorado Académico

Proyecto de carrera: Ingeniería en Informática

Asignatura: Ingeniería del Software II

Estimación en Entornos Complejos

Integrantes	C.i
Arnaldo Perdomo	30.791.551
Daniel Reyna	29.989.066
Jhoanny Maita	30.694.732
Yoryelis Ocando	31.074.651

Puerto Ordaz, 10 de Septiembre del 2026

Estimación en Entornos Complejos

1. Resolución del ejercicio

Enunciado:

De la Incertidumbre (PERT) a la Realidad Empírica (Monte Carlo / Flujo)
Contexto: Su equipo debe estimar el desarrollo de un módulo de autenticación biométrica. Un arquitecto senior, usando su experiencia, da la siguiente estimación de tres puntos para el esfuerzo total en días-persona: Optimista = 10, Más Probable = 20, Pesimista = 60.

Calcule la estimación de esfuerzo esperado (T_e) utilizando la fórmula de PERT. ¿Por qué la media ponderada da un resultado tan diferente al Más Probable?
Giro Ágil: En lugar de usar días-persona, el equipo decide usar Story Points. Tras 3 Sprints históricos, el equipo tiene un Throughput (velocidad) promedio de 30 Story Points por Sprint, con una desviación estándar de 5. Si el backlog del módulo tiene 90 Story Points, utilice la lógica de las métricas de flujo para explicar cómo calcularía la fecha de entrega y por qué esto es más confiable que la estimación de PERT.

El cálculo PERT y la trampa del "Más Probable"

Datos del arquitecto:

- Optimista (O) = 10 días-persona
- Más Probable (M) = 20 días-persona
- Pesimista (P) = 60 días-persona

Fórmula PERT:

$$T_e = \frac{O + 4M + P}{6}$$

Sustituyendo:

$$T_e = \frac{10 + 4(20) + 60}{6} = \frac{10 + 80 + 60}{6} = \frac{150}{6} = 25 \text{ días-persona}$$

¿Por qué la media ponderada (25) es diferente al valor "Más Probable" (20)?

Aquí está el truco. El "Más Probable" es solo el valor que el experto cree que tiene más chances de ocurrir, como la moda de una distribución. Pero el PERT no se queda solo con ese valor, porque en el desarrollo de software casi nunca las cosas salen "bien" nomás.

El valor **Pesimista (60)** está muy lejos, y aunque no es lo más común, el PERT le da un peso importante en el cálculo (junto con los 4 pesos del "Más Probable"). Como la distribución del esfuerzo en software suele tener una **cola larga hacia la derecha** (siempre aparecen imprevistos, dependencias, bugs ocultos), el promedio ponderado se "corre" hacia arriba. Básicamente, el PERT nos dice: "Oye, el escenario más probable son 20, pero el riesgo de que se vaya a 60 es tan grande que, en promedio, mejor espera 25". Es una forma de no ser demasiado optimistas.

El giro Ágil y el poder de los datos reales (Flujo)

Datos del equipo:

- Backlog del módulo = 90 Story Points (SP).
- Velocidad promedio (Throughput) en los últimos 3 Sprints = 30 SP por Sprint.

- Desviación estándar de esa velocidad = 5 SP.

Cálculo simple (el promedio):

Si mantenemos el ritmo, el número de Sprints esperados sería:

$$\frac{90 \text{ SP}}{30 \frac{\text{SP}}{\text{Sprint}}} = 3 \text{ Sprints}$$

Pero ojo, aquí viene lo bonito del enfoque de flujo:

No nos quedamos con un número fijo, sino que usamos la desviación estándar para calcular un **rango de confianza**.

Si la velocidad varía ± 5 SP por Sprint, y vamos a necesitar 3 Sprints para completar 90 SP, la desviación estándar total se calcula así (asumiendo que los Sprints son independientes):

$$\sigma_{total} = \sqrt{3} * 5 \approx 1.75 * 5 \approx 8.66 \text{ SP}$$

Esto significa que, estadísticamente:

- **Rango del 68% (1 desviación):** el esfuerzo real estará entre 90 ± 8.66 SP, o sea, entre 81.34 y 98.66 SP. Traducido a Sprints: entre **2.7 y 3.3 Sprints**.
- Si queremos ser más precavidos (digamos, un 85% de confianza, como menciona el texto), usaríamos un factor Z de ~ 1.04 , lo que nos da un rango de Sprints de aproximadamente **2.9 a 3.6 Sprints**.

¿Por qué esta estimación con flujo es MÁS confiable que el PERT?

Acá va mi análisis como estudiante:

1. **El PERT es una "foto" subjetiva:** Depende completamente de lo que un arquitecto *cree* que va a pasar. Es un voto de confianza a la memoria y criterio de una sola persona. Además, los días-persona son una métrica engañosa porque no reflejan los impedimentos del día a día (reuniones, context switching, esperar revisiones).
2. **El Flujo (Velocidad) son datos duros de la casa:** No estamos adivinando, estamos midiendo *lo que el equipo ya hizo* en los últimos 3 Sprints. Esa velocidad de 30 SP ya incluye todo el "ruido" real: los bugs, las deudas técnicas, las distracciones, las reuniones y la dinámica del equipo.
3. **El PERT da una falsa precisión:** Decir "25 días" suena muy exacto, pero en la práctica es un volado. El enfoque de flujo, usando la desviación estándar, nos permite decir: *"Lo más probable es que terminemos en 3 Sprints, pero si la cosa se pone fea, podemos tardar hasta 3.6 Sprints"*. Esto nos ayuda a gestionar las expectativas de los stakeholders sin mentirles.
4. **Es auto-ajutable:** A medida que pasen los Sprints, puedo recalcular la velocidad y la desviación con datos más frescos. En cambio, el PERT se quedó fijo desde el día 1, sin importar si el equipo encontró un problema grave o una librería que les ahorró la mitad del trabajo.

Conclusión final del ejercicio:

La fórmula PERT me da **25 días-persona** como "esperanza matemática", pero el enfoque ágil me dice que, basado en hechos reales, el equipo entregará en **~ 3 Sprints** (con un margen de error de medio Sprint para arriba o abajo).

Prefiero mil veces la segunda opción, porque está basada en el rendimiento real del equipo, no en una corazonada de un experto (por muy senior que sea).

2. Resolución del ejercicio

Enunciado:

La Falacia de los Recursos y el Cono de Incertidumbre Contexto: Faltan 2 meses para la fecha límite de un proyecto de software crítico. El proyecto está retrasado. El Director de la Empresa, ignorando la Ley de Brooks, ordena duplicar el equipo de desarrollo (de 5 a 10 programadores) para recuperar el tiempo perdido y cumplir con el cronograma de Gantt original. Analice matemáticamente y conceptualmente por qué la ecuación $\text{Tiempo} = \text{Esfuerzo} / \text{Recursos}$ falla en este escenario específico.

a) Detalle, explique tres (3) factores de sobrecarga (comunicación, onboarding, integración) que ocurrirán al duplicar el equipo.

b) Proponga, explique dos (2) alternativas de gestión (una desde un enfoque predictivo tradicional y otra desde un enfoque ágil/híbrido) para manejar esta crisis de cronograma sin violar los principios de la ingeniería de software.

a) Por qué la ecuación Tiempo igual a Esfuerzo entre Recursos falla

Esta fórmula asume que el trabajo de programar se puede dividir entre las personas, como si fuera cargar ladrillos: si una persona tarda diez días, diez personas deberían tardar un día. Pero programar no es así. Antes de que alguien nuevo pueda ayudar, tiene que entender el proyecto, y eso toma tiempo de las personas que ya están trabajando.

Esto es lo que describe la Ley de Brooks: agregar personal a un proyecto de software que ya está atrasado lo atrasa todavía más. La razón es que el tiempo que ganan los recursos nuevos trabajando se pierde varias veces en explicarles el proyecto, coordinar su trabajo con el resto del equipo y corregir los conflictos que surgen al integrar código escrito por más gente.

También está el Cono de Incertidumbre: al principio de un proyecto no se sabe bien cuánto va a tardar, y ese margen de error se reduce con el tiempo, a medida que el equipo aprende más. En este caso el proyecto ya va avanzado. Meter gente nueva ahora no reduce lo que falta por saber, lo aumenta, porque agrega cosas nuevas justo cuando queda menos tiempo para corregir errores.

Los tres factores de sobrecarga al duplicar el equipo

- **Comunicación:** Mientras más personas hay, más conversaciones hacen falta para que todos sepan lo mismo, y esas conversaciones crecen muy rápido. Con cinco personas hay diez pares que necesitan hablarse. Cada conversación es una oportunidad de que algo se entienda mal o alguien se quede sin saber algo importante. El equipo termina gastando más tiempo coordinándose que programando.
- **Onboarding:** Un programador que se suma al proyecto no puede aportar valor de inmediato. Primero necesita entender la arquitectura del sistema, las decisiones de diseño ya tomadas, las convenciones del equipo y el estado actual del código. Ese proceso de adaptación consume tiempo tanto del nuevo integrante como de los miembros experimentados, que deben pausar

su propio trabajo para explicar el proyecto. En un contexto donde el plazo ya está comprometido, ese tiempo de enseñanza es tiempo que se resta directamente a la fecha de entrega.

- **Integración:** Más personas trabajando sobre el mismo código al mismo tiempo significa más cambios que después hay que combinar. Esto eleva la probabilidad de conflictos al fusionar código, de que una persona modifique algo que otra estaba usando de otra forma, y de que aparezcan errores que solo se detectan cuando las piezas se juntan. Coordinar estas integraciones, revisar los cambios de más gente y resolver los conflictos que surgen consume tiempo adicional que no existía cuando el equipo era más pequeño.

b) Alternativas de Gestión ante la Crisis de Cronograma

1. Alternativa: Enfoque Predictivo Tradicional

- **Reevaluar el alcance (MoSCoW).** En vez de sumar personal, se clasifican las funcionalidades pendientes en Must have, Should have, Could have y Won't have. Solo los requisitos "Must have" se protegen para la fecha límite; el resto se negocia o se traslada a una entrega posterior. Esto ataca la causa real del problema demasiado alcance para el tiempo disponible en lugar de intentar comprar tiempo agregando personas.
- **Negociar fechas.** Se presenta al director un cronograma actualizado y realista, respaldado por el diagrama de Red/CPM vigente, mostrando con datos objetivos por qué la fecha original ya no es alcanzable con el equipo actual. Es preferible comprometerse con una nueva fecha realista y cumplirla, que sostener la fecha original y volver a fallar.
- **Añadir recursos estratégicamente (no duplicar).** Si se decide incorporar personal, se hace de forma quirúrgica: una o dos personas como máximo, asignadas a tareas acotadas y desacopladas del núcleo del sistema (documentación, pruebas, módulos independientes), nunca en la ruta crítica ni en componentes que exigen conocimiento profundo del diseño existente.
- **Priorizar tareas críticas.** Utilizando el diagrama de Red/CPM se identifican las tareas que se encuentran en la ruta crítica, y se concentra allí el esfuerzo del equipo actual, posponiendo temporalmente las tareas secundarias que no bloquean la entrega.

2. Alternativa: Enfoque Ágil/Híbrido

- **Reducir alcance (slicing).** En lugar de intentar entregar funcionalidades completas, estas se dividen en "rebanadas" más pequeñas que aportan valor por sí solas (vertical slicing). Así se logra entregar una versión funcional, aunque reducida, en vez de arriesgarse a no completar nada.
- **Entregas incrementales.** Se define un producto mínimo viable (MVP) que cubra lo esencial y se entrega primero, dejando mejoras y funcionalidades secundarias para iteraciones posteriores. Esto le da al director algo tangible y funcional en la fecha límite, en lugar de un esquema de todo o nada.
- **Kanban / Flow.** Se visualiza el trabajo en un tablero Kanban para detectar cuellos de botella en tiempo real y se limita el trabajo en progreso (WIP), evitando que el equipo se disperse en múltiples tareas simultáneas justamente lo que se agrava al duplicar personal sin coordinación.
- **Replanificación con datos reales.** Se utiliza el Throughput histórico del equipo (Story Points o tareas completadas por semana) para recalcular, con datos reales y no con optimismo, cuánto del backlog es alcanzable en los dos meses restantes, comunicando esa proyección con transparencia al director.

Conclusión

Ambas alternativas evitan el error de la Ley de Brooks: en lugar de aumentar los recursos humanos sobre un cronograma fijo, ajustan el alcance o el ritmo de entrega a la capacidad real del equipo, apoyándose en datos objetivos (CPM, MoSCoW, Throughput) en vez de decisiones impulsivas que agravan la sobrecarga de comunicación, onboarding e integración.